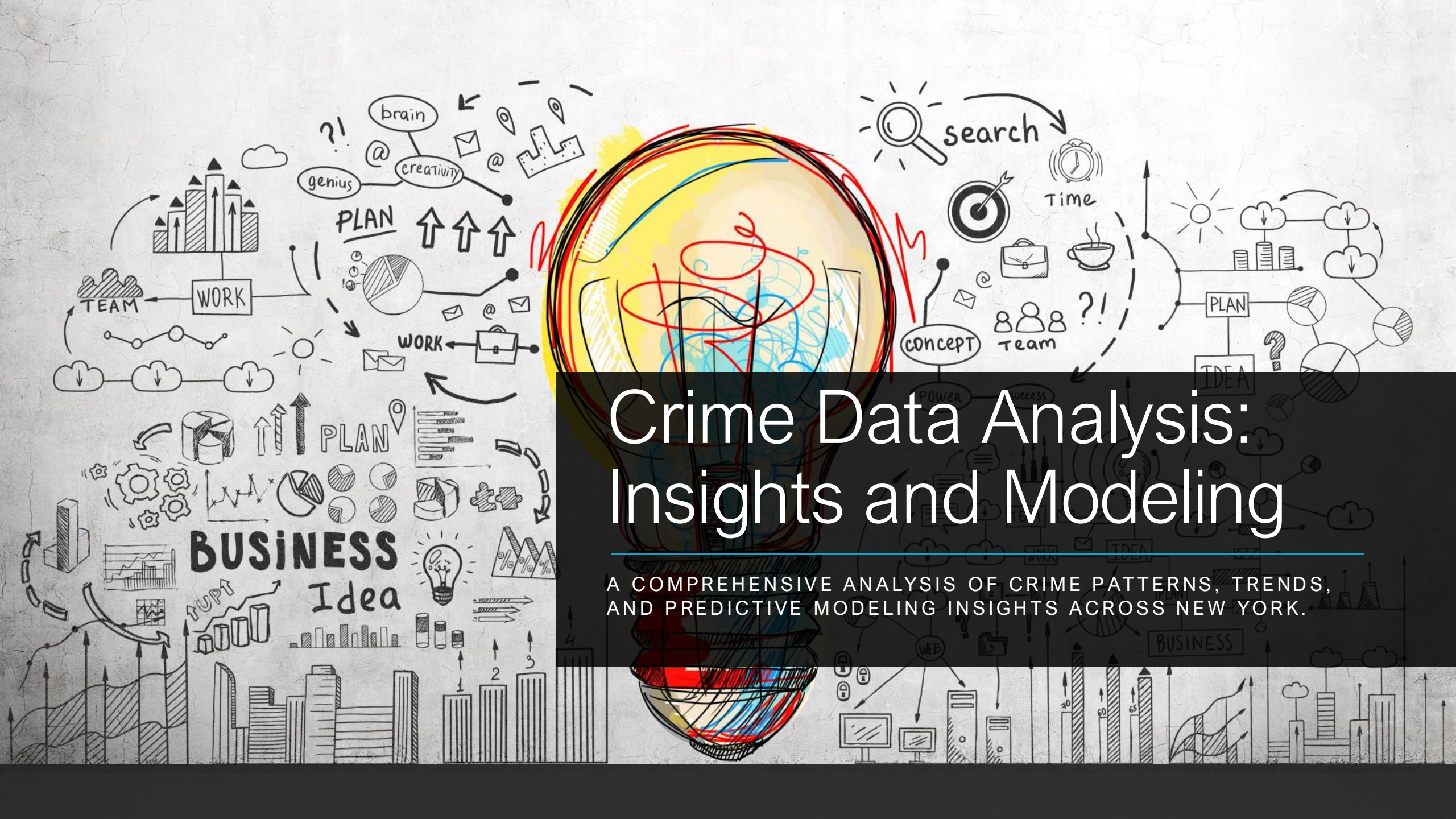




Crime Data Analysis: Insights and Modeling

A COMPREHENSIVE ANALYSIS OF CRIME PATTERNS, TRENDS,
AND PREDICTIVE MODELING INSIGHTS ACROSS NEW YORK.



Introduction

This presentation provides an overview of the analysis conducted on crime data from various counties and agencies in New York. It highlights key insights, methodologies employed, and the implications of the findings for understanding crime patterns and trends.

Loaded Dataset

The dataset was loaded into a pandas DataFrame for analysis, specifically the file named `Index_Crimes_by_County_and_Agency__Beginning_1990_20251008.csv`.

Initial Data Inspection

Initial inspection methods such as `head()`, `tail()`, `info()`, and `describe()` were utilized to understand the data structure, column types, and basic statistics, revealing valuable insights about the dataset.

```
In [7]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 23830 entries, 0 to 23829
Data columns (total 15 columns):
#   Column                Non-Null Count  Dtype
---  -
0   County                23830 non-null  object
1   Agency                23830 non-null  object
2   Year                  23830 non-null  int64
3   Months Reported       14134 non-null  float64
4   Index Total           23830 non-null  int64
5   Violent Total         23830 non-null  int64
6   Murder                23830 non-null  int64
7   Rape                  23830 non-null  int64
8   Robbery               23830 non-null  int64
9   Aggravated Assault    23830 non-null  int64
10  Property Total        23830 non-null  int64
11  Burglary              23830 non-null  int64
12  Larceny               23830 non-null  int64
13  Motor Vehicle Theft   23830 non-null  int64
14  Region                23830 non-null  object
dtypes: float64(1), int64(11), object(3)
memory usage: 2.7+ MB
```

```
In [77]: df.shape
```

```
Out [77]: (23830, 15)
```

Dataset Overview

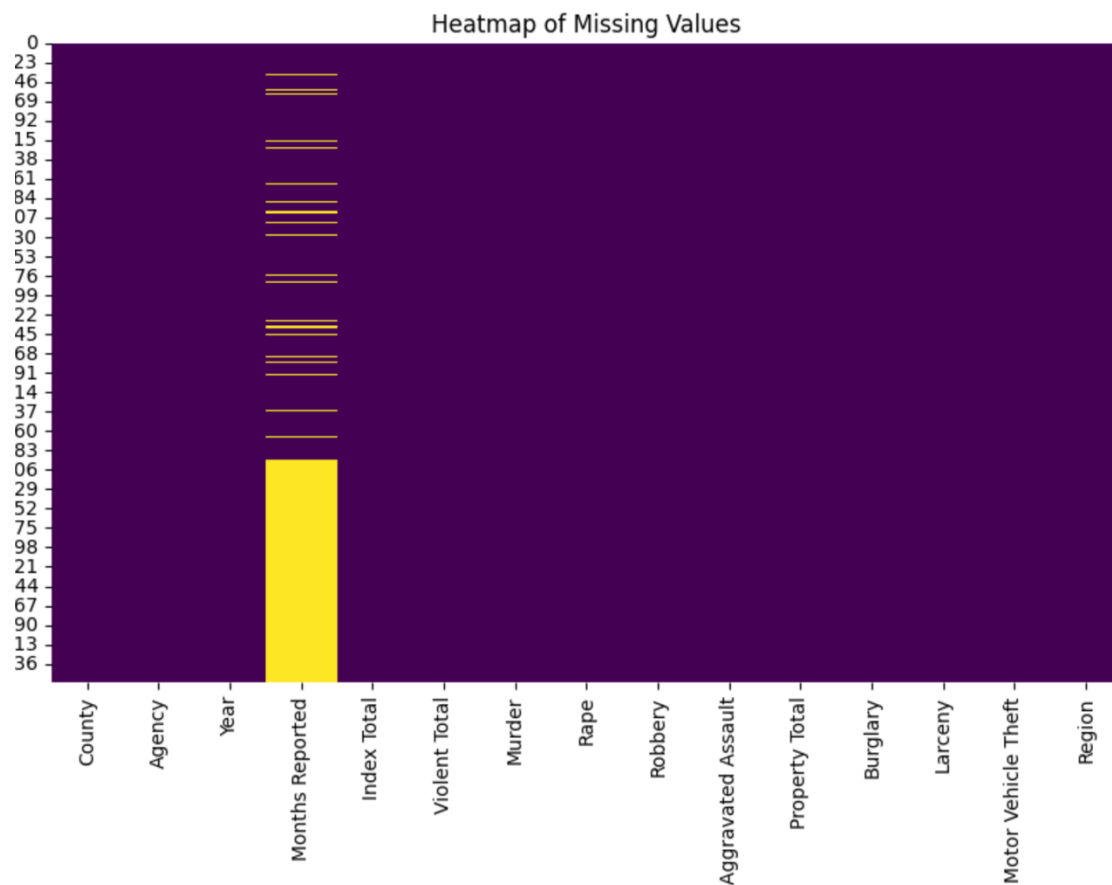
```
In [11]: df.isnull().sum()
```

```
County          0
Agency         0
Year            0
Months Reported 9696
Index Total     0
Violent Total   0
Murder          0
Rape            0
Robbery         0
Aggravated Assault 0
Property Total  0
Burglary        0
Larceny         0
Motor Vehicle Theft 0
Region          0
dtype: int64
```

Data Preprocessing and Cleaning

CHECK THE NULL VALUE

```
figure(figsize=(10, 6))
heatmap(df.isnull(), cbar=False, cmap='viridis')
title('Heatmap of Missing Values')
show()
```



CHECKING MISSING VALUE WITH HEATMAP

Impute missing value with median

Impute missing values

Replace the missing values in the 'Months Reported' column with the calculated median.

```
In [22]: median_months_reported = df['Months Reported'].median()
df['Months Reported'].fillna(median_months_reported, inplace=True)
```

/tmp/ipython-input-3186158377.py:2: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.

```
df['Months Reported'].fillna(median_months_reported, inplace=True)
```

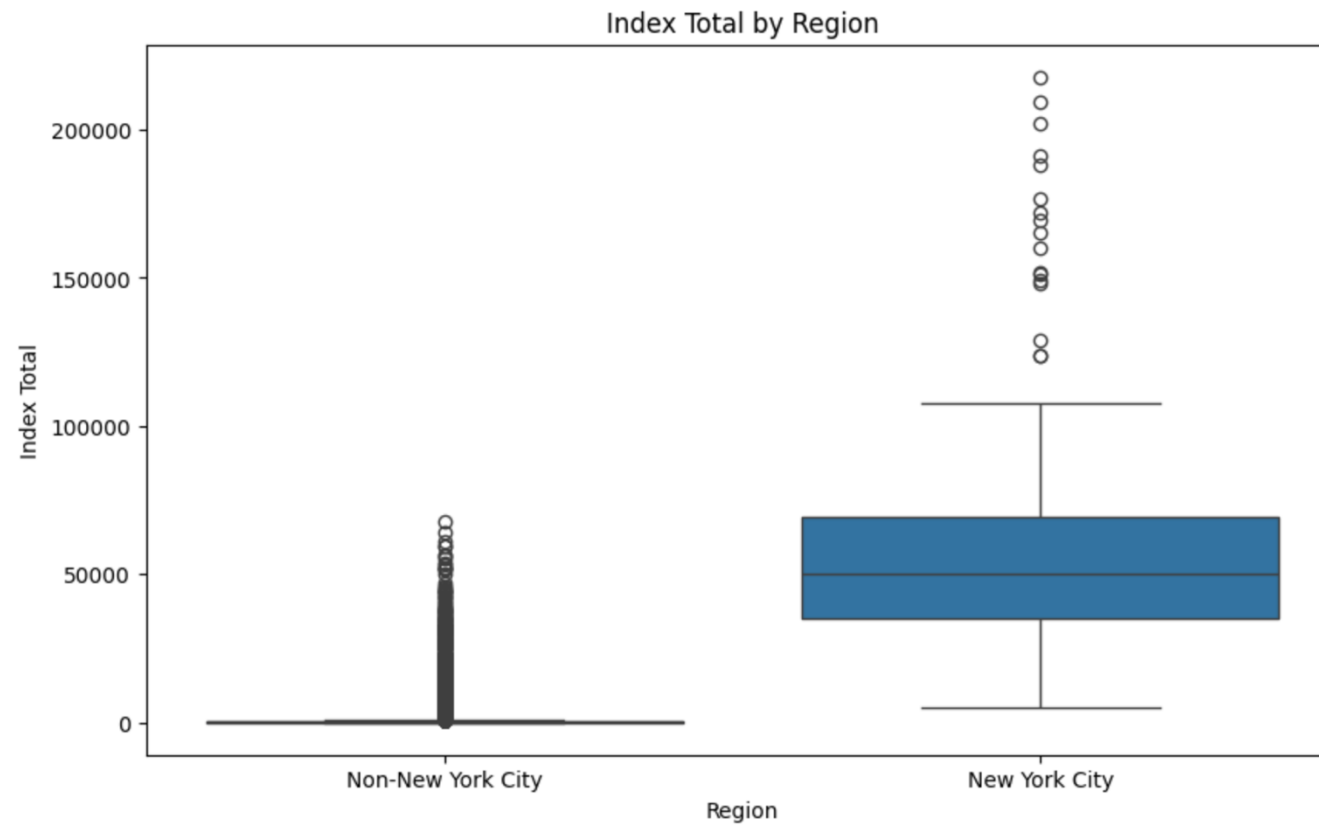
Verify imputation

```
In [23]: df.isnull().sum()
```

```
County          0
Agency         0
Year            0
Months Reported 0
Index Total     0
Violent Total   0
Murder          0
Rape           0
Robbery         0
Aggravated Assault 0
Property Total  0
Burglary        0
Larceny         0
Motor Vehicle Theft 0
Region          0
dtype: int64
```

Boxplot of Index total by Region

```
In [14]: #2. Box plot of 'Index Total' by 'Region'
plt.figure(figsize=(10, 6))
sns.boxplot(x='Region', y='Index Total', data=df)
plt.title('Index Total by Region')
plt.xlabel('Region')
plt.ylabel('Index Total')
plt.show()
```



Standardize values

```
In [19]: # Example: Standardize variations of 'Police Department' to 'PD'
df['Agency'] = df['Agency'].replace({
    ' Police Dept': ' PD',
    ' Police Department': ' PD',
    ' P.D.': ' PD',
    ' Sheriff\'s Office': ' Sheriff',
    ' S.O.': ' Sheriff'
})

# Re-print the unique values for the modified 'Agency' column to verify standardization
print("Unique values in column 'Agency' after standardization:")
print(df['Agency'].unique())
print("-" * 30)
```

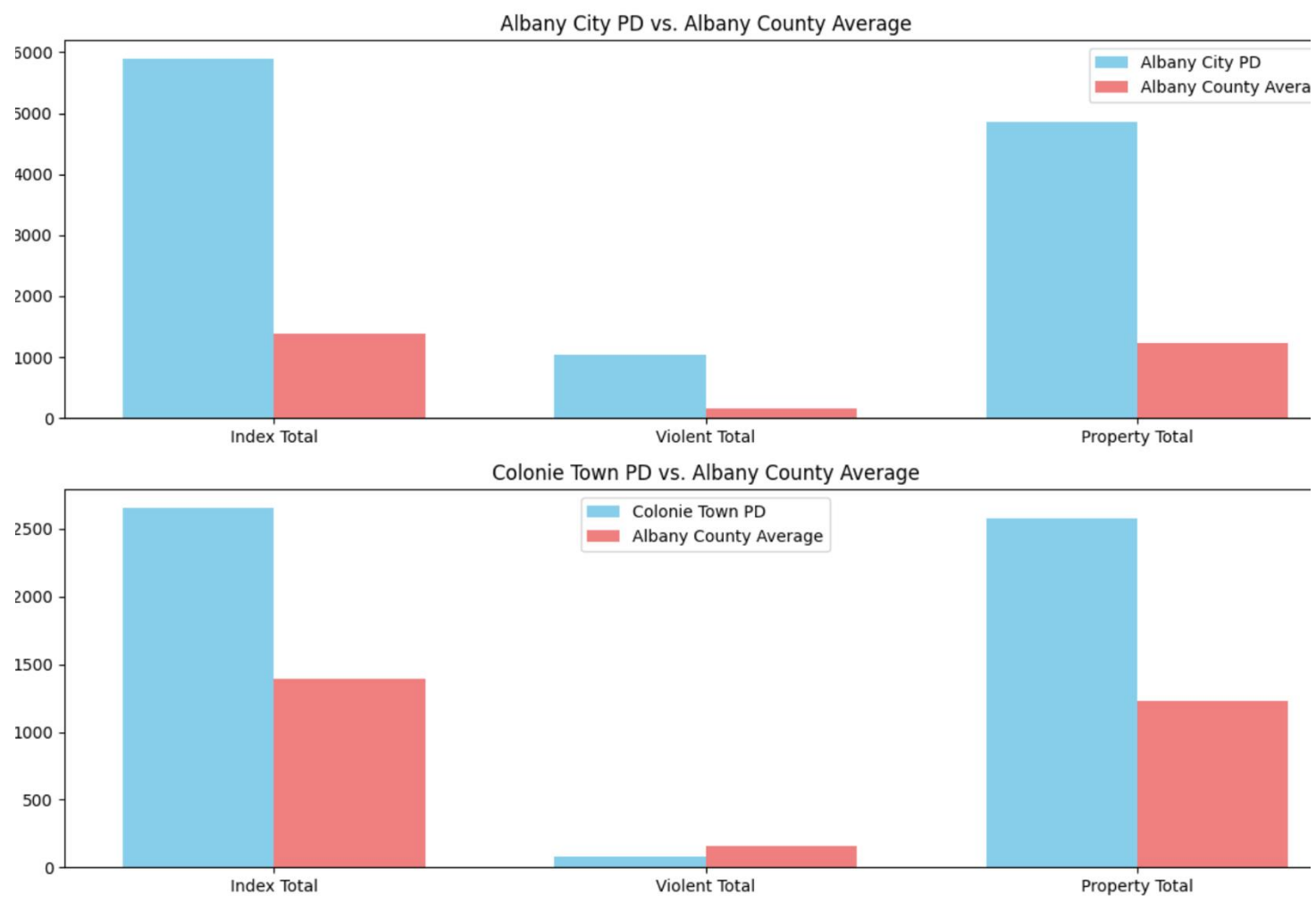
```
Unique values in column 'Agency' after standardization:
['Albany City PD' 'Albany County Park PD' 'Albany County Sheriff'
 'Albany County State Police' 'Altamont Vg PD' 'Bethlehem Town PD'
 'Coeymans Town PD' 'Cohoes City PD' 'Colonie Town PD' 'County Total'
 'Guilderland Town PD' 'Menands Vg PD' 'SUNY Police - Albany'
 'SUNY Police - SUNY Plaza' 'Watervliet City PD' 'Alfred Vg PD'
 'Allegany County Park PD' 'Allegany County State Police' 'Andover Vg PD'
 'Belmont Vg PD' 'Bolivar Vg PD' 'Cuba Town PD' 'Friendship Town PD'
 'Independence Town PD' 'SUNY Police - Alfred' 'Wellsville Vg PD'
 'Willing Town PD' 'Binghamton City PD' 'Broome County Park PD'
 'Broome County Sheriff' 'Broome County State Police' 'Endicott Vg PD'
 'Johnson City Vg PD' 'Port Dickinson Vg PD' 'SUNY Police - Binghamton'
 'Vestal Town PD' 'Allegany Vg PD' 'Cattaraugus County Park PD'
 'Cattaraugus County Sheriff' 'Cattaraugus County State Police'
 'Cattaraugus Vg PD' 'Ellicottville Town PD' 'Franklinville Vg PD'
 'Olean City PD' 'Portville Vg PD' 'Salamanca City PD' 'Auburn City PD'
 'Cayuga County Park PD' 'Cayuga County Sheriff'
 'Cayuga County State Police' 'Moravia Vg PD' 'Port Byron Vg PD'
 'Weedsport Vg PD' 'Carroll Town PD' 'Chautauqua County Park PD'
 'Chautauqua County Sheriff' 'Chautauqua County State Police'
 'Dunkirk City PD' 'Ellicott Town PD' 'Fredonia Vg PD' 'Jamestown City PD'
 'Lakewood-Busti PD' 'SUNY Police - Fredonia' 'Westfield Vg PD'
 'Chemung County Park PD' 'Chemung County Sheriff'
 'Chemung County State Police' 'Elmira City PD' 'Elmira Heights Vg PD'
 'Elmira Town PD' 'Afton Vg PD' 'Bainbridge Vg PD'
 'Chenango County Park PD' 'Chenango County Sheriff']
```

Task 1

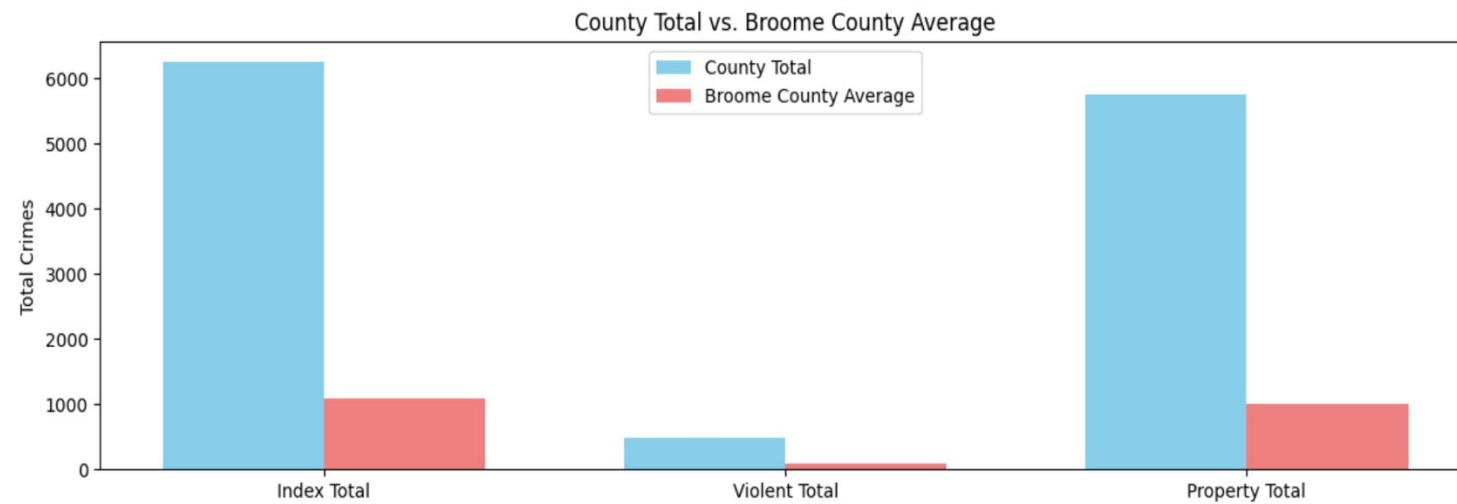
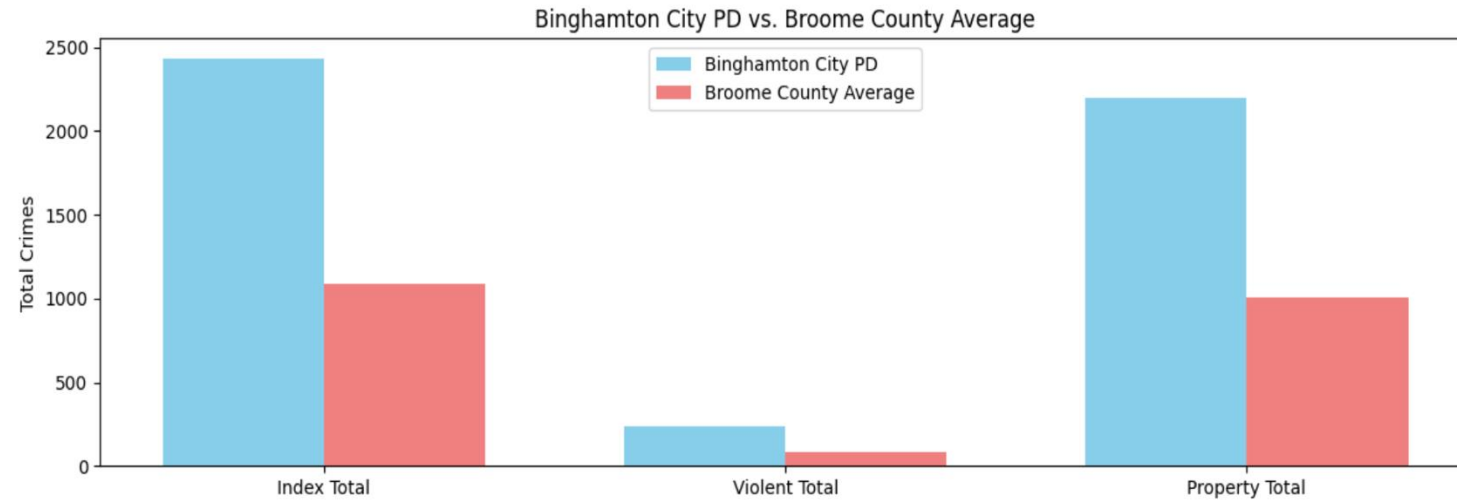
Problem Statement 1: Identify agencies with atypical crime reporting patterns compared to their respective counties.

####Bar plots for each county in atypical_agencies comparing agency crime totals to county crime totals.

Crime Totals: Agencies vs. County Average in Albany



Crime Totals: Agencies vs. County Average in Broome



Prepare data for regression

Select relevant features, handle categorical variables (e.g., using one-hot encoding), and split the data into training and testing sets.

```
In [48]: from sklearn.model_selection import train_test_split

# 1. Select relevant features
features = [
    'Year', 'Months Reported', 'Violent Total', 'Property Total', 'Murder',
    'Rape', 'Robbery', 'Aggravated Assault', 'Burglary', 'Larceny',
    'Motor Vehicle Theft', 'County', 'Agency', 'Region'
]
target = 'Index Total'

# 2. Handle categorical variables using one-hot encoding
df_encoded = pd.get_dummies(df[features], columns=['County', 'Agency', 'Region'], drop_first=True)

# 3. Define the feature matrix X and the target vector y
X = df_encoded
y = df[target]

# 4. Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

print("Shape of X_train:", X_train.shape)
print("Shape of X_test:", X_test.shape)
print("Shape of y_train:", y_train.shape)
print("Shape of y_test:", y_test.shape)

Shape of X_train: (19064, 915)
Shape of X_test: (4766, 915)
Shape of y_train: (19064,)
Shape of y_test: (4766,)
```

```
In [49]: from sklearn.ensemble import RandomForestRegressor

# Instantiate the model
model = RandomForestRegressor(random_state=42)

# Train the model
model.fit(X_train, y_train)
```

```
Out [49]: RandomForestRegressor
```

Problem Statement for Regression:

Predict the 'Index Total' crime count for a given agency in a specific year, based on historical crime data, agency characteristics, and potentially regional information. This would involve using features like 'Year', 'Months Reported', 'Violent Total', 'Property Total', and potentially encoded categorical features like 'County', 'Agency', and 'Region' to build a regression model that predicts the continuous numerical value of 'Index Total'.

```
In [50]: from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
```

```
# Make predictions on the test set
y_pred = model.predict(X_test)

# Calculate evaluation metrics
mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)

# Print the results
print(f"Mean Absolute Error (MAE): {mae:.2f}")
print(f"Mean Squared Error (MSE): {mse:.2f}")
print(f"Root Mean Squared Error (RMSE): {rmse:.2f}")
print(f"R-squared (R2): {r2:.2f}")
```

```
Mean Absolute Error (MAE): 19.05
Mean Squared Error (MSE): 68012.61
Root Mean Squared Error (RMSE): 260.79
R-squared (R2): 1.00
```

Result

```
In [79]: print("Comparison of Regression Model Performance:")
print("-" * 50)
print("Metric          | Linear Regression | Ridge Regression | RandomForestRegressor")
print("-" * 50)
print(f"MAE             | {mae_linear:<17.2e} | {mae_ridge:<16.2e} | {mae:.2f}")
print(f"MSE             | {mse_linear:<17.2e} | {mse_ridge:<16.2e} | {mse:.2f}")
print(f"RMSE            | {rmse_linear:<17.2e} | {rmse_ridge:<16.2e} | {rmse:.2f}")
print(f"R2              | {r2_linear:<17.2f} | {r2_ridge:<16.2f} | {r2:.2f}")
print("-" * 50)
```

Comparison of Regression Model Performance:

Metric	Linear Regression	Ridge Regression	RandomForestRegressor
MAE	2.97e-12	6.25e-07	19.05
MSE	4.67e-23	7.31e-12	68012.61
RMSE	6.84e-12	2.70e-06	260.79
R2	1.00	1.00	1.00

Problem Statement for Classification:

Classify the 'Region' (New York City or Non-New York City) based on the reported crime numbers for a given agency and year. This would involve using the various crime count columns ('Index Total', 'Violent Total', 'Murder', 'Rape', etc.) as features to train a classification model that predicts the categorical variable 'Region'.

##Task 2

#####Analyze Problem Statement 2: Trends in Violent vs. Property Crimes in New York City vs. Non-New York City Regions.

Prepare data for classification

```
In [60]: from sklearn.model_selection import train_test_split

# 1. Select relevant features (crime count columns)
features = [
    'Index Total', 'Violent Total', 'Murder', 'Rape', 'Robbery',
    'Aggravated Assault', 'Property Total', 'Burglary', 'Larceny',
    'Motor Vehicle Theft'
]
X = df[features]

# 2. Select the target variable ('Region')
y = df['Region']

# 3. Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 4. Print the shapes of the resulting training and testing sets
print("Shape of X_train:", X_train.shape)
print("Shape of X_test:", X_test.shape)
print("Shape of y_train:", y_train.shape)
print("Shape of y_test:", y_test.shape)

Shape of X_train: (19064, 10)
Shape of X_test: (4766, 10)
Shape of y_train: (19064,)
Shape of y_test: (4766,)
```

####Train a Logistic Regression model and a RandomForestClassifier model on the training data.

```
In [61]: from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier

# Instantiate and train Logistic Regression model
logistic_model = LogisticRegression(random_state=42, max_iter=1000) # Increased max_iter for convergence
logistic_model.fit(X_train, y_train)

# Instantiate and train RandomForestClassifier model
rf_model = RandomForestClassifier(random_state=42)
rf_model.fit(X_train, y_train)

print("Logistic Regression model trained.")
print("Random Forest Classifier model trained.")
```

Logistic Regression model trained.
Random Forest Classifier model trained.

```
In [73]: from sklearn.tree import DecisionTreeClassifier

# Instantiate and train Decision Tree model
dt_model = DecisionTreeClassifier(random_state=42)
dt_model.fit(X_train, y_train)

print("Decision Tree Classifier model trained.")
```

Decision Tree Classifier model trained.

Result

Logistic Regression Model Evaluation:

Accuracy: 1.00

Classification Report:

	precision	recall	f1-score	support
New York City	1.00	0.97	0.99	35
Non-New York City	1.00	1.00	1.00	4731
accuracy			1.00	4766
macro avg	1.00	0.99	0.99	4766
weighted avg	1.00	1.00	1.00	4766

Confusion Matrix:

```
[[ 34  1]
 [  0 4731]]
```

Random Forest Classifier Model Evaluation:

Accuracy: 1.00

Classification Report:

	precision	recall	f1-score	support
New York City	1.00	0.97	0.99	35
Non-New York City	1.00	1.00	1.00	4731
accuracy			1.00	4766
macro avg	1.00	0.99	0.99	4766
weighted avg	1.00	1.00	1.00	4766

Confusion Matrix:

```
[[ 34  1]
 [  0 4731]]
```

Decision Tree Model Evaluation:

Accuracy: 1.00

Classification Report:

	precision	recall	f1-score	support
New York City	0.97	0.94	0.96	35
Non-New York City	1.00	1.00	1.00	4731
accuracy			1.00	4766
macro avg	0.99	0.97	0.98	4766
weighted avg	1.00	1.00	1.00	4766

Confusion Matrix:

```
[[ 33  2]
 [  1 4730]]
```